基于信息可逆嵌入的图像认证与自修复系统

赵罗曼，216001943

摘要：本项目旨在实现一个具备图像篡改认证、篡改定位以及篡改后自修复的系统整体。系统整体采用分块后对每个图像块进行备份信息概括，并将备份信息嵌入到指定的映射块中的方法，当某图像块受到篡改后，可以从其映射块中提取备份信息进行恢复。为提高不可恢复现象出现的可能性，本文提出对图像进行关联块绑定的方式，从而实现一个块的备份信息存在于多个块之中，提早了容灾能力；对于图像映射块的选择方式，本文采用多种方式，最终引入密钥来确定映射块，保证映射块之间的距离较远且难以被破解。最后，本文基于 PyQT 进行了用户界面设计，使算法实现与软件的业务接口有效对接，提高用户的使用体验。

关键词: 可逆水印；图像篡改认证；篡改自修复；基于密钥的映射；PyQT 界面设计
中图分类号:　　文献标识码:

随着信息技术的日新月异，数字图像已成为我们日常生活与工作的核心信息媒介，其重要性不言而喻。然而，在图像的传输、存储及应用环节中，它们面临着多样化的篡改与攻击风险，严重威胁到图像信息的真实面貌与完整性。因此，如何高效且可靠地保障数字图像的信息安全，确保其内容未经篡改且完整无损，已成为信息安全领域亟待攻克的关键课题。传统图像认证手段大多依赖于密码学原理，如加密技术与数字签名，这些方法虽能有效保护图像信息，但往往需对整个图像进行加密或签名处理，这不仅降低了处理效率，还限制了对图像内容的局部验证与自我恢复能力。为解决这一瓶颈，本项目创新性地提出了基于可逆水印嵌入技术的图像认证与自修复系统方案，旨在实现对数字图像的高效局部认证与智能自修复功能，从而克服传统方法的局限性。

此系统的成功开发不仅具有深远的现实意义，它能够有效提升图像信息的安全防护水平，降低图像处理过程中的成本消耗，进一步推动数字图像在各领域的广泛应用与发展，同时也为信息安全技术的持续进步贡献力量。综上所述，本项目所倡导的基于可逆水印嵌入的图像认证与自修复系统，不仅在理论上具有重大突破，更在实际应用中展现出极高的价值，为数字图像信息安全防护开辟了新的路径与策略。

1 方案实现流程简述

对于《基于可逆水印的信息嵌入与自修复系统》，其系统功能的功能模块包含图像加密、图像篡改认证、图像自修复三个部分。对于图像加密，需要对图像进行分块处理，并对每一个图像块进行备份信息的计算，为了实现篡改后的自修复，需要将该备份信息嵌入至别的图像块中；而对于图像的自修复过程，则只需要按照上述加密的过程进行分块，由于每个图像块嵌入信息都经过了 Hash 的拼接处理，因此可以使用 Hash 的重计算与对比进行篡改认证，若存在受到篡改现象，则可以从别的图像块中提取备份信息进行恢复操作。

具体地，加密过程包含以下步骤：

1. **图像分割策略**：将原始图像分割成多个尺寸为 d×d 的图像块，每个独立的图像块将作为后续进行篡改检测、定位及自修复操作的基本单元。
2. **提取图像块备份信息**：为了确保图像在遭受篡改后能够进行有效自修复，需从每个图像块中提取关键信息作为备份。这些信息需高度压缩，以控制其大小，如限制在 32 位二进制数内，以便高效存储与传输。
3. **备份信息的编码与篡改检测**：为了赋予备份信息篡改检测的能力，采用计算哈希值并适当拼接的方法对提取的信息进行编码。这样，任何对原始图像块的篡改都将导致对应哈希值的变化，从而被系统检测到。
4. **映射块的精心选择**：针对编码后的备份信息，需选择合适的图像块（称为映射块）进行嵌入。鉴于图像篡改往往发生在连续区域内，需要确保映射块与原始图像块之间的距离足够远，以减少同时被篡改的风险。
5. **引入关联块机制**：鉴于单一图像块的备份信息可能在篡改事件中受损，采用关联块策略。通过组合两个图像块的信息进行联合编码并嵌入，可显著降低备份信息单独受损的可能性，增强系统的鲁棒性。
6. **可逆隐写技术的应用**：利用可逆隐写技术将编码后的备份信息嵌入到映射块中。这一步骤可采用多种算法实现，如基于差值扩展的可逆隐写算法或基于直方图平移的可逆隐写算法，确保信息嵌入的同时保持图像质量不受明显影响，且后续能够无损恢复原始图像。

图像的篡改认证与自修复过程包含以下若干步骤：

1. **图像分割**：将图像按照在加密过程中采用的相同分块策略进行分割，确保每个图像块都是独立且均匀划分的。
2. **构建校验矩阵**：为了追踪图像块的完整性，建立一个校验矩阵（Check Matrix）。通过遍历每个图像块，提取其中嵌入的信息，并将其分解为原始数据和哈希码两部分。随后重新计算每个块的哈希值，并与存储的原始哈希码进行比较。如果两者一致，说明该图像块未被篡改，在校验矩阵中对应位置标记为 1；若不一致，则标记为 0，表示该块已遭受篡改。
3. **认证结果可视化**：为了直观展示图像认证的结果，采用黑白图像块的方式。具体地，对于校验矩阵中标记为 0 的位置（即被篡改的图像块），在结果图像中相应位置用黑色块表示，以清晰指示哪些区域存在问题。
4. **篡改图像自修复**：针对那些被确认为篡改的图像块（即校验矩阵中为 0 的部分），尝试从其预设的映射块中提取备份信息以进行恢复。若不幸发现映射块本身也被篡改（即其在校验矩阵中的标记同样为 0），则我们进一步转向关联块对应的映射块，尝试从中提取备份信息。一旦成功获取到有效的备份信息，将立即进行图像的自修复过程，以尽可能恢复原始图像内容。

分析上述过程可知，如果出现图像块及嵌入其信息的映射块都受到了篡改，则该图像就无法实现恢复，故本文着重于探究如何提高图像的容灾能力，即降低上述图像及其备份信息都受到篡改的可能性。即对于加密过程中的第 4、5 两点进行深入研究，除此之外，本项目，将各功能模块进行集成，并基于 PyQT 进行了可视化的用户界面设计，以保证上述模型的可用性与便捷性。

2 容灾能力提升方案

2.1 关联块的选定

一种基于直觉的方案为：若某个图像的块的信息嵌入到了其它一个图像块中，则当该图像块与嵌入其信息的图像块都受到了篡改，则无法实现图像修复，故考虑将备份信息嵌入到多个不同的图像块，这样会使得无法恢复的情况变为多个图像块均受到了篡改攻击，这种事件的发生概率相对于单备份场景来说更小，从而降低了图像无法修复的概率。因此本文考虑将多个块的信息进行拼接操作，并将拼接结果作为每个块的备份信息，通过这种操作可以使得每个信息都被存储于多个地方，且拼接后处理 Hash 往往具备更小的空间开销。

为简化实现流程，本文采用将两个相邻的图像块的备份信息进行拼接，通过将相邻块组成关联块，进行信息映射，即使块 1 的映射块受损，依然可以依靠块 2 的映射块恢复信息，这种冗余的设计提高了系统对局部损坏的容忍度。此外，当一幅图像的某个局部区域遭受损坏时，根据图像的像素间的空间相关性，可以预见到其邻近区域也很可能受到损害。由于这种损坏的局部性和扩散性，进行图像修复时可以同时对相邻块进行修复，提高了修复效率。

整体示意图如下图 2.1 所示，即将相邻块作为关联块进行信息拼接，嵌入到对应的映射块中。

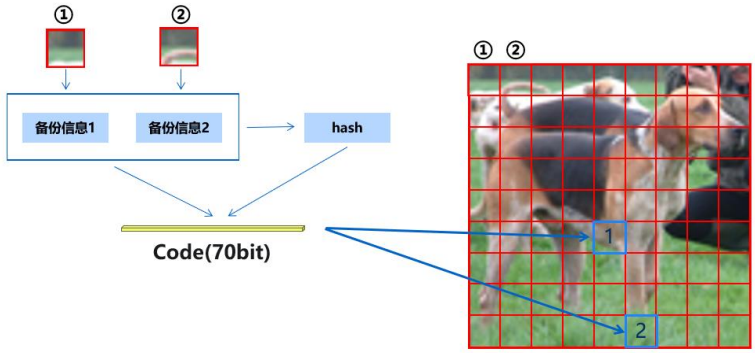


图 2.1 图像的关联块选定

2.2 映射块的确定

一个图像的映射块是唯一的，即为该图像的备份信息所嵌入的块，对于映射块的选定，需要保证以下若干特征：

（1）确定性

即每个图像的映射块是唯一确定的，且能够进行还原，以便于图像认证过程中的映射块寻找。这要求映射过程需要通过一个确定的函数进行确定，而不能够使用随即映射方法。

（2）距离的保持

对于图像及其映射块，需要保证两个图像块的距离较远，从而减少图像局部受损带来的连带影响，而若需要满足每一对映射块之间的距离都是最大的，这是一个 NP-Hard 问题，当图像块数目较多时，无法在有限的时间内进行求解于处理，故本文首先考虑一

个最简单的映射方法，如下所示，其中 Y 为映射块的标号，X 为原图像块的标号，N 为总的图像块的数目。从直观上理解该式子，即对每个图像块都进行了半张图片的偏移，这使得从整体来看映射块之间具有较远的距离。

$$Y=(X+\frac{N}{2})\bmod\ N$$

（3）安全性

上述映射方法存在一个缺点，即每个图像块的映射块很容易被确定，在知道图像块总数的情况下，能够直接找到一个图像块的映射块，这便给攻击提供了便利，即同时攻击图像及其映射块便能够造成无法恢复，缺乏一定的安全性。针对该问题，本文引入了两个密钥进行映射块的确定，映射方法如下所示。

$$Y=(\ (K_1\times X)\bmod\ N+K_2)\bmod\ N$$

这种方法引入了两个密钥进行了映射的控制，使得映射过程相对难以被破解，如下图 3.2 所示，即为引入密钥的部分映射过程，可以看到，映射过程较难被预测，且也保证了上述“距离的保持”的特性要求。但这也要求了对于同一张图像的加解密需要使用同一对密钥。

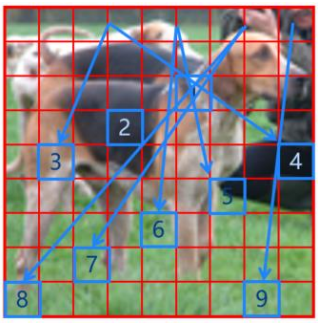


图 2.2 引入密钥的映射过程

（4）避免冲突

对于上述包含密钥的映射过程，可能存在碰撞的可能性，即对于两个不同的 X，结果得到一个相同的映射块 Y，这会导致两个块的备份信息同时嵌入到一个块中，并直接导致某一个块的映射信息被覆盖，即产生了信息丢失，这对图像的自修复功能是灾难性的影响，故避免冲突是必要的环节。

考虑到每个块的映射块是唯一的，因此初始块的集合大小与映射块的集合大小必定相同，且为原始图像块的数目，因此每个图像块总能够找到其映射块，故可以考虑使用开放定址法解决碰撞问题，这导致程序引入了额外的数组空间，但也在可接受范围之内。除此之外，可以考虑引入密钥仓库，根据具体的图形块数目 N，在多个可能的密钥对中寻找不会产生冲突的密钥对。

3 系统总体设计

本系统包含图像加密、图像篡改认证、图像自修复三大功能模块，在本次课程设计中将其集成为了一个软件整体，并设计了直观、简洁且用户友好的交互界面，确保用户能够轻松上手，快速完成图像认证与修复操作。

3.1 软件模块内容

本系统采用模块化设计，将核心功能划分为四个主要模块：图像加密模块、图像的篡改认证与定位模块、图像的自修复模块和操作日志模块。各模块之间通过标准接口进行交互，确保系统的可扩展性和可维护性。

1. **图像加密模块**：图像加密模块负责对输入的原始图像进行加密处理，采用先进的加密算法对图像数据进行编码，生成加密后的图像。加密过程保证图像数据的安全性，防止未经授权的访问和篡改。
2. **图像的篡改认证与定位模块**：篡改认证与定位模块是本系统的核心模块之一。它首先通过可逆水印技术将认证信息嵌入到加密后的图像中，形成水印图像。当接收到可能被篡改的图像时，该模块能够提取出水印信息，并与原始认证信息进行比对，判断图像是否被篡改。若发生篡改，该模块能够定位到具体的篡改区域，为后续的篡改分析和处理提供依据。
3. **图像的自修复模块**：自修复模块利用可逆水印技术中的冗余信息，对篡改后的图像进行自修复。当检测到图像被篡改时，该模块会根据水印信息中的冗余数据，对篡改区域进行恢复，尽可能还原原始图像的内容。自修复过程无需人工干预，大大提高了处理效率和准确性。
4. **操作日志模块**：操作日志模块负责记录系统运行过程中的所有操作信息，包括图像加密、篡改检测、自修复等操作的时间、图像名称与内容、操作结果等详细信息。这些信息对于后续的审计、分析和故障排查具有重要价值。操作日志模块采用安全可靠的存储机制，确保日志信息的安全性和完整性。

3.2 工作流程

从用户角度来对系统的整体工作流程进行概述。流程如下：

1. 用户通过界面输入原始图像，启动图像加密模块进行加密处理，同时系统满足多个图像同时加密的需求。

2. 加密后的图像通过可逆水印技术嵌入认证信息，形成水印图像，用户可选择保存图像。

3. 当需要验证图像是否被篡改时，用户将待验证的图像输入系统，启动篡改认证与定位模块进行检测。

4. 若检测到图像被篡改，用户可以通过自修复模块对篡改区域进行恢复，系统将输出恢复后的图像，用户可选择保存图像。

5. 在整个系统运行过程中，操作日志模块持续记录所有操作信息，以供后续审计和分析使用。

3.3 系统结构框图

根据系统模块内容，可确定系统的大致结构，如下图 3.1 所示。

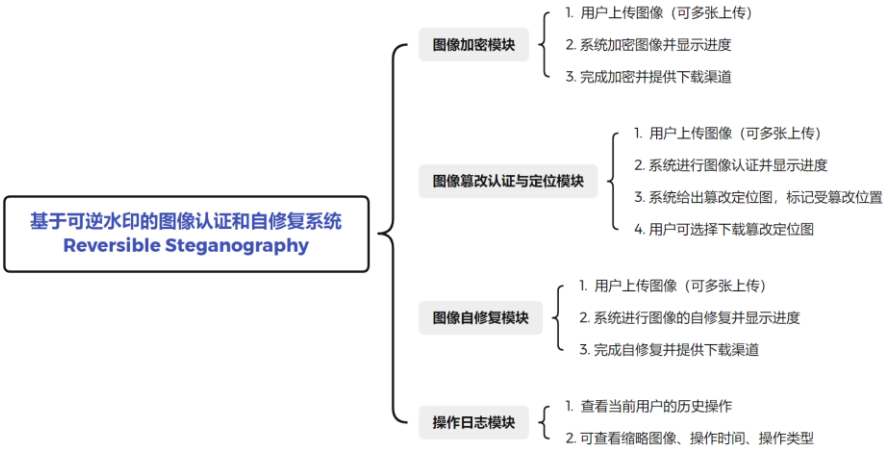


图 3.1 系统结构框图

4 用户 UI 界面设计

4.1 界面设计概要

本次课程设计中，遵循了简洁、直观、一致性等 UI 设计原则，确保用户能够快速理解程序的使用方法，并提高操作效率。本程序采用 Python 语言开发，并使用 PyQt5 作为 GUI 框架，实现了一个稳定且功能丰富的用户界面。界面采用了清晰的颜色对比，以蓝色和白色为主色调，配合直观的图标设计，增强了视觉效果和用户识别度。除此之外，程序界面主要由菜单栏、工具栏、状态栏和操作区组成。菜单栏提供了程序的主要功能入口，工具栏包含了常用操作的快捷方式，状态栏显示当前操作的状态信息，操作区则是用户进行具体操作的地方。

4.2 功能模块

下文将分别对每个模块的界面和使用进行介绍。

4.2.1 登陆界面

登陆界面用于保护用户数据和设置，确保只有授权用户能够访问程序功能，登陆界面如下图 4.1 所示。



图 4.1 系统登录界面运行图

4.2.2 主页

主页作为程序的中心枢纽，需要确保良好的交互性，提供快速访问所有主要功能的入口；除此之外，主页作为用户对一个软件的第一印象的标准，主页的美观性也是不可或缺的，在主页我们可以直观的看到相关数据，主页界面如下图 4.2 所示

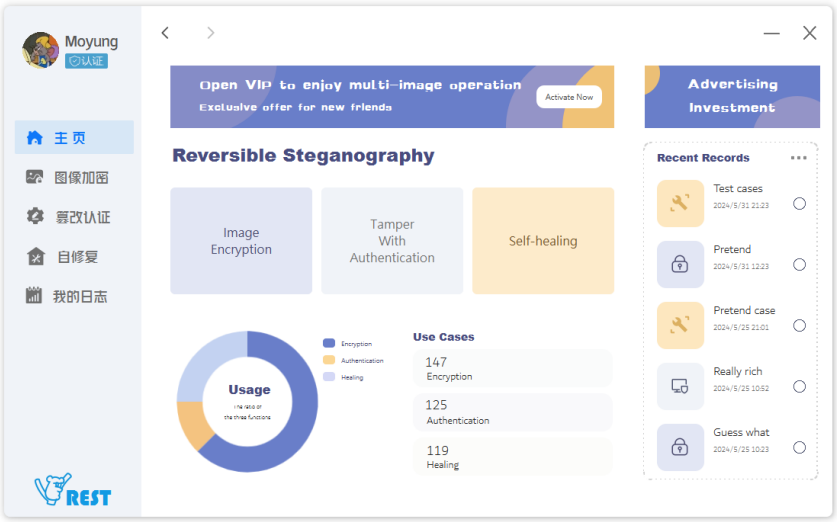


图 4.2 系统主页界面运行图

4.2.3 图像加密

用户可以通过点击主页按钮或侧边栏来进入嵌入水印的界面。该界面提供图像加密功能，确保图像内容的安全性，并且供图像的略缩图和下载。如下图 4.3-1 至图 4.3-3 所示为用户使用图像加密的流程，用户通过点击按钮，选择想要加密的图片，本软件提供一次性对多张彩色图片进行水印嵌入；在程序运行水印嵌入代码时，我们采用加载条的方式更加直观地显示水印嵌入的进度；水印嵌入完成后，我们提供对该图片的缩略图用于预览，并且用户可以通过点击下载图篇下载想要保存的已嵌入水印的图片。



图 4.3-1 待加密界面运行图

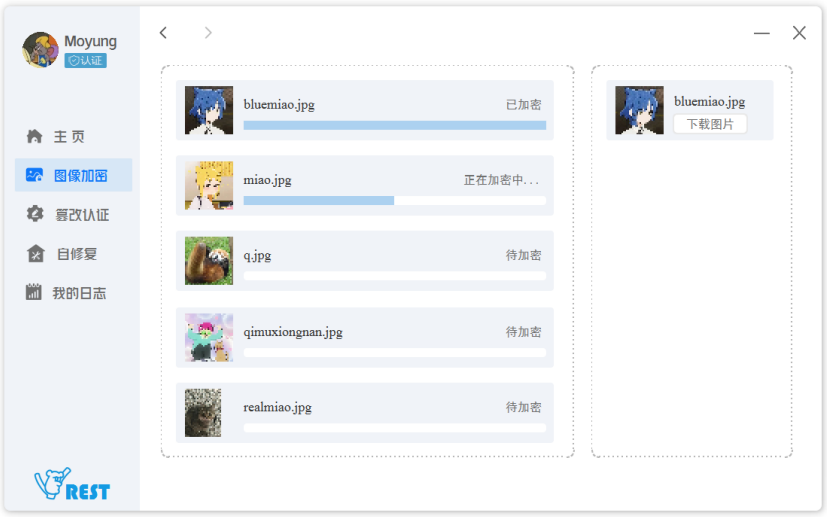


图 4.3-2 加密过程运行图

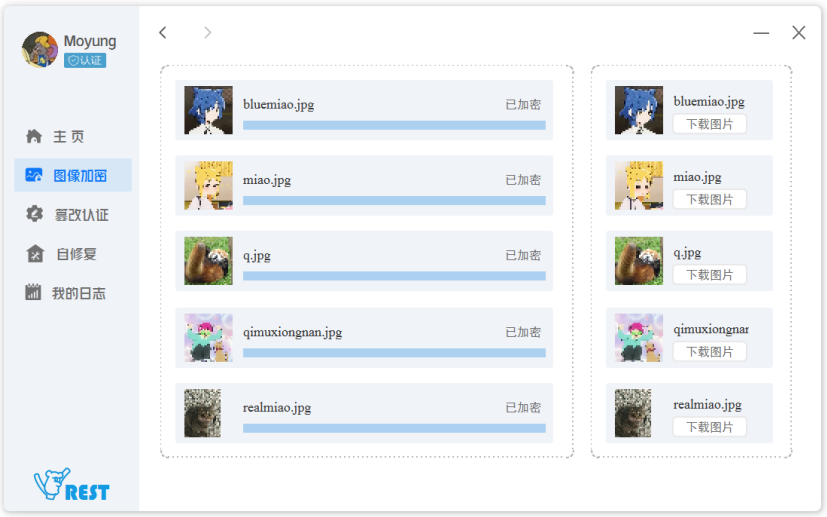


图 4.3-3 加密完成运行图

4.2.4 图像篡改认证

用户可以通过点击主页按钮或侧边栏来进入篡改认证的界面。该界面提供图像篡改认证功能，检测图像是否被篡改，并提供认证信息。如下图 4.4 为用户使用篡改认证功能的过程，用户通过点击按钮，选择想要进行篡改认证的图片；在程序运行篡改认证代码时，我们采用与水印嵌入相同的加载条的方式，更加直观地显示篡改认证的进度；篡改认证完成后，我们提供对该图片的认证信息的缩略图用于预览，并且用户可以通过点击下载图篇下载想要保存的图片认证信息。

4.2.5 图像自修复

用户可以通过点击主页按钮或侧边栏来进入图像自修复的界面。该界面提供图像自修复功能，当检测到输入的图像损坏时，进行修复操作。图像自修复功能的操作流程与水印嵌入和篡改认证无异，如下图 4.5 为用户使用图像自修复功能的过程。

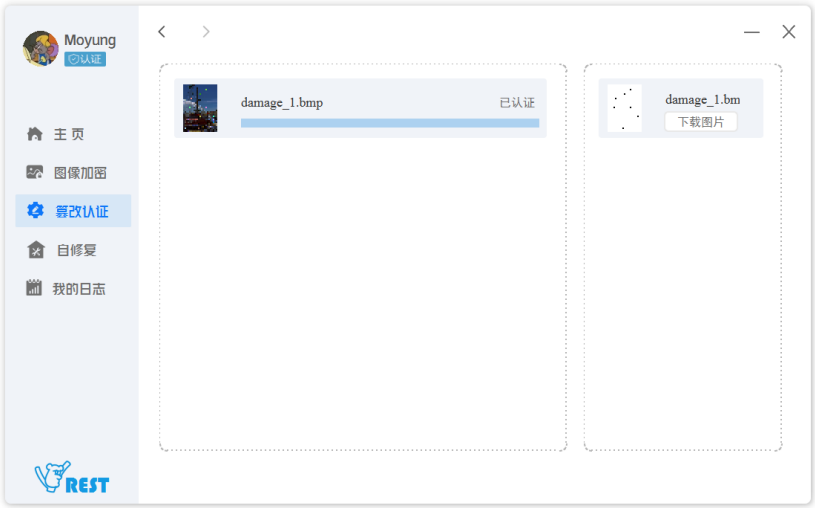


图 4.4 图像篡改认证运行图

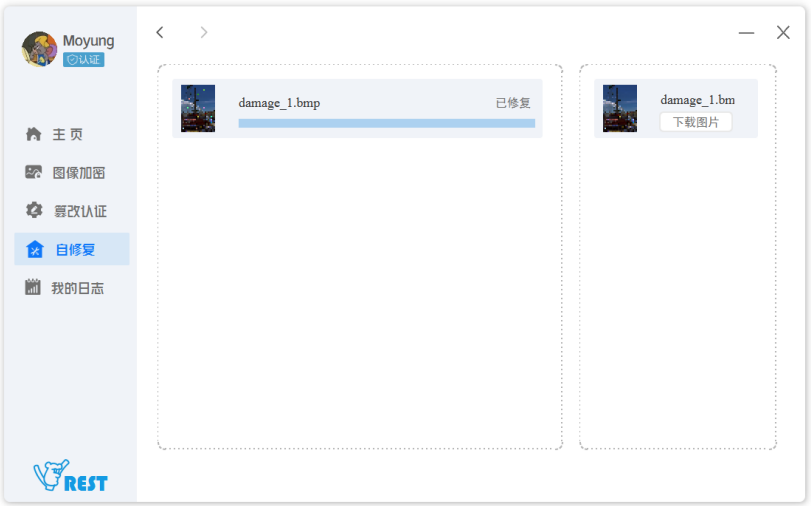


图 4.5 图像自修复运行图

4.2.6 系统使用日志

用户可以通过点击主页按钮或侧边栏来进入使用日志的界面。该界面记录用户的操作历史，便于问题追踪和系统审计。在该界面中，用户可以看到历史操作记录的信息，包括时间、图片、图片名和操作类型。并且为了方便寻找相关记录，该界面提供根据时间、图片名和操作类型来进行升降序。如下图 4.6 所示。

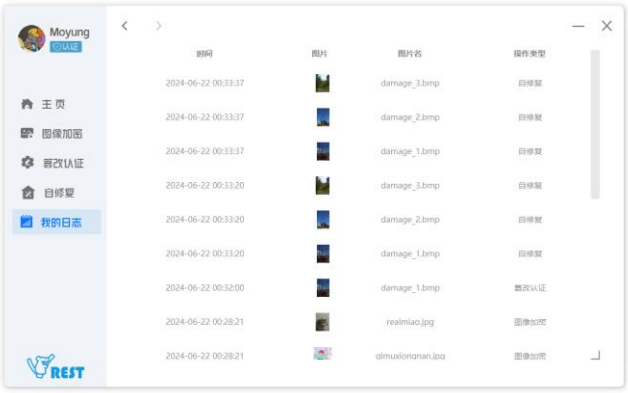


图 4.7 系统使用日志

5 软件效果与功能测试

5.1 图像加密功能测试

进入软件“图像加密”模块，可以选择图像，如下图 4.3-1 所示。通过下图所示的文件浏览器打开待加密的图像，这里选定 1.png。

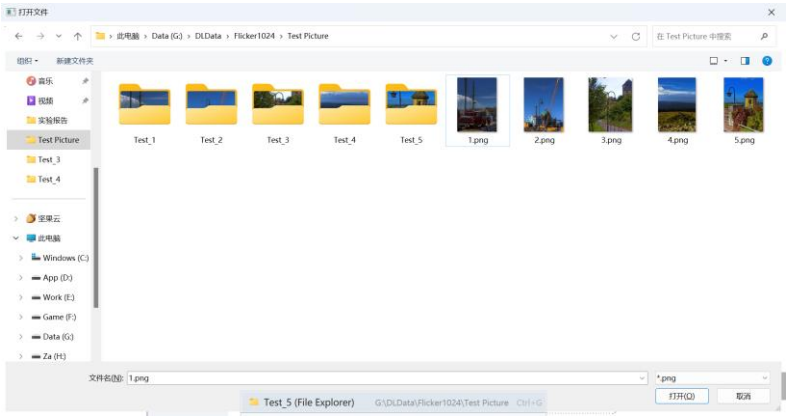


图 5.1 图像选择

图像上传至软件并等待加密，如下图 5.2 所示。

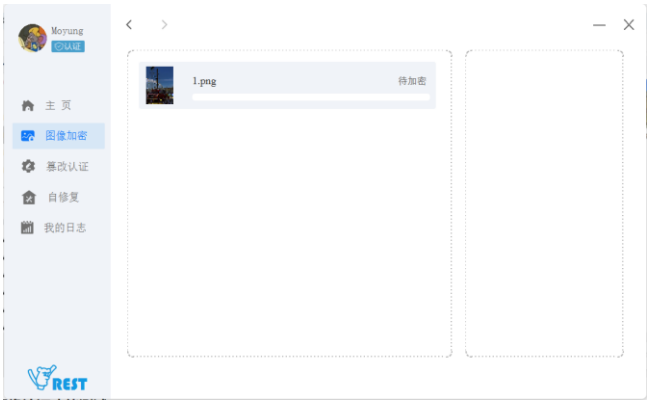


图 5.2 图像加密

图像完成加密后，提供下载链接，可以点击“下载图像”并将图片保存至本地，如图 5.3 所示。

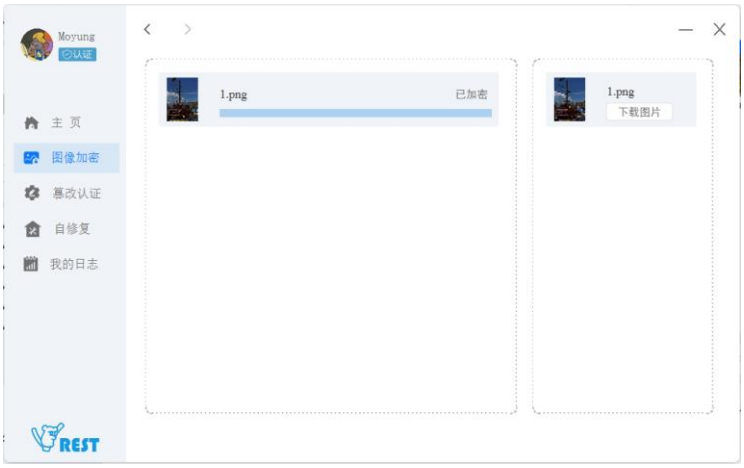


图 5.3 图像加密完成

下载加密后的图像，将其命名为 emb_1.png，如下图 5.4 所示。

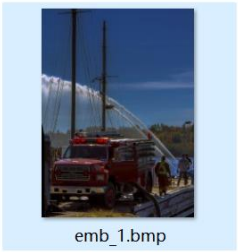


图 5.4 加密完成后的图片

5.2 图像完整性认证功能测试

进入软件“篡改认证”模块，可以选择图像。选择一张受到篡改的图像，为测试软件，采用随机生成噪声块的方式进行图像的篡改。过程如下图 5.5 所示，该受到篡改的图像文件名为（damage_1.bmp）。



图 5.5 图像噪声篡改

将篡改图上传软件进行篡改定位。如下图 5.6 所示，软件正在进行图像的认证。

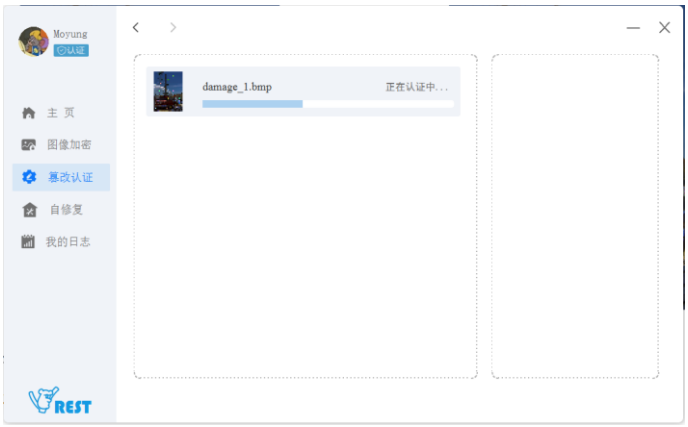


图 5.6 图像认证过程

完成认证后，与图像加密模块一致，软件提供下载链接，下载软件输出的篡改定位图，命名为 loc_1.bmp，如下图 5.7 所示。可以看到篡改定位图中成功对受到篡改的部位进行了定位。

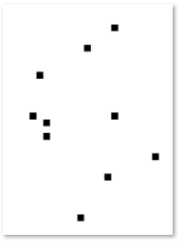


图 5.7 图像篡改定位图

5.3 图像自修复功能测试

进入软件“篡改认证”模块，可以选择图像，将图像 damage_1.bmp 上传至软件自修复功能模块。如下图 5.8 所示，软件正在进行图像的修复。

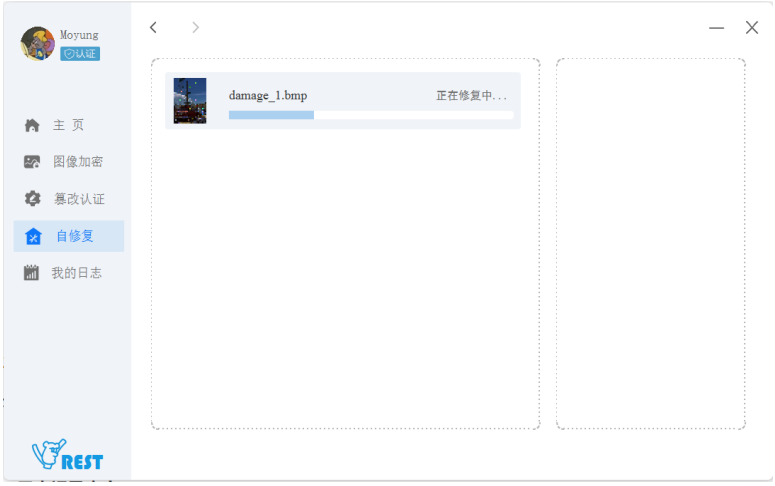


图 5.8 图像篡改定位图

完成修复后，与图像加密模块一致，软件提供下载链接，下载软件输出的修复图像，命名为 fix_1.bmp，如下图 5.9 所示。



fix_1.bmp

图 5.9 图像自修复结果

与原图进行对比，如下图 5.10 所示，可以看到篡改均得到了修复，修复质量取决于索引库的质量与训练数据集的内容。

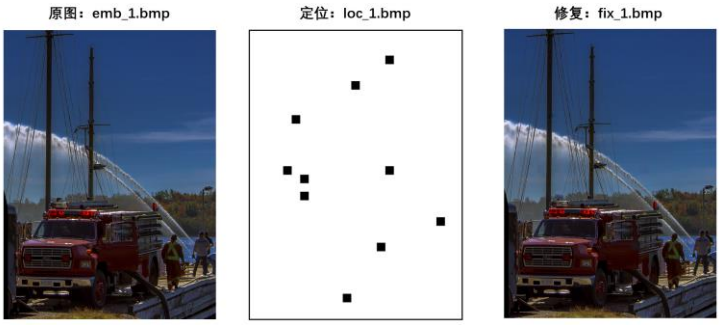


图 5.10 图像认证定位与自修复效果对比

6 总结与展望

在《基于可逆水印的图像认证与自修复系统》的研发旅程中，我深入探索了图像映射的精细过程，从确定性、距离性、安全性、防碰撞性等角度触发，引入密钥对实现映射过程的控制，并成功将理论转化为实践应用。此外，基于以实现的图像加密与自修复等功能模块，我利用 PyQT 精心构建的用户交互界面，不仅提升了系统的易用性，设计了直观、简洁且用户友好的交互界面，确保用户能够轻松上手，快速完成图像认证与修复操作。

在项目中算法的设计阶段，我致力于分析已有问题，逐步寻找问题的解决方法以找到符合项目开发需求的技术方法，如探索安全有效的图像映射方法以精确映射图像特征至可逆水印中，同时确保图像在受到篡改之后出现无法恢复的可能性大大降低。最后，通过 PyQT 框架的采用，则为系统披上了一层友好的外衣，通过直观的操作界面，用户可以轻松上传图像、查看认证结果，并在必要时触发自修复功能，恢复图像的原貌。通过不懈的努力与多次迭代优化，该系统成功实现了对图像特征的精准概括与可逆嵌入，即使在图像遭遇恶意篡改的情况下，也能凭借内置的备份信息快速识别篡改区域并进行高质量的自动修复。此外，利用 PyQT 设计的界面不仅美观大方，还极大提升了用户体验，使得非专业人员也能轻松上手操作。

除此之外，我深刻认识到，在图像认证与自修复领域，仍有许多未知与挑战等待我们去探索。比如，进一步优化图像特征的提取与压缩策略，减少备份信息的存储负担；拓展系统对多种图像尺寸与分辨率的支持，以适应更广泛的应用场景；以及加强系统的安全性设计，确保备份信息的绝对安全。总的来说，通过此次实践，我深刻体会到了理论与实践相结合的重要性，以及持续学习与创新的必要性。这次课程设计不仅是对我专业知识的综合运用，更是对未来研究方向的一次深刻洞察与前瞻。

参考文献:

[1] Kiran S.Mohanty, Koustuv S. Das. Image Processing, Analysis, and Machine Vision (3rd Edition). CRC Press, 2016.

[2] Tian, J. (2003). Reversible data embedding using a difference expansion. IEEE Transactions on Circuits and Systems for Video Technology, 13(8), 890-896.

[3] B. Chen and G. W. Wornell, "Quantization index modulation: a class of provably good methods for digital watermarking and information embedding", IEEE Trans. Inf. Theory, vol. 47, no. 4, pp. 1423-1443, May 2001.

[4] Baluja, S. (2017). Hiding images in plain sight: Deep steganography. Advances in Neural Information Processing Systems, 30.

[5] Tang, W., Tan, S., & Wang, B. (2018). End-to-end trained CNN encoder-decoder networks for image steganography. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (pp. 2497-2506).

[6] Zhu, J.-Y., Kaplan, R., Johnson, J., & Fei-Fei, L. (2018). Invisible steganography via generative adversarial network. In Proceedings of the European Conference on Computer Vision (ECCV) (pp. 214-229).

[7] Kingma, D. P., & Welling, M. (2013). Auto-encoding variational bayes. arXiv preprint arXiv:1312.6114.

[8] Eric Matthes. Python Crash Course: A Hands-On, Project-Based Introduction to Programming. 2019

[9] Kiran S.Mohanty, Koustuv S. Das. Image Processing, Analysis, and Machine Vision (3rd Edition). CRC Press, 2016.

Image Authentication and Self-Recovery System Based on Reversible Information Embedding

Abstract: This project aims to realize an integrated system capable of image tampering authentication, tampering localization, and self-recovery after tampering. The system employs a method of dividing the image into blocks, summarizing backup information for each block, and embedding this information into designated mapping blocks. When a specific image block is tampered with, its backup information can be extracted from its mapping block for recovery. To reduce the likelihood of irrecoverable scenarios, this paper proposes a method of associating blocks, ensuring that the backup information of one block exists in multiple blocks, thereby enhancing disaster tolerance capabilities. Regarding the selection of image mapping blocks, multiple approaches are explored, ultimately incorporating a key to determine the mapping blocks, ensuring that they are far apart and difficult to decipher. Finally, this paper designs a user interface based on PyQt, effectively bridging the algorithm implementation with the software's business interface, thereby enhancing the user experience.

Key words: Reversible Watermarking; Image Tampering Authentication; Self-Recovery from Tampering; Key-Based Mapping; PyQt Interface Design

CLC number: **Document code:**